

- [CPSC 375: High-Performance Computing](#)

[CPSC 375: High-Performance Computing](#)

Spring 2026

- [Syllabus](#)
- [Schedule](#)
- [Upload](#)
- [The Cluster Project](#)

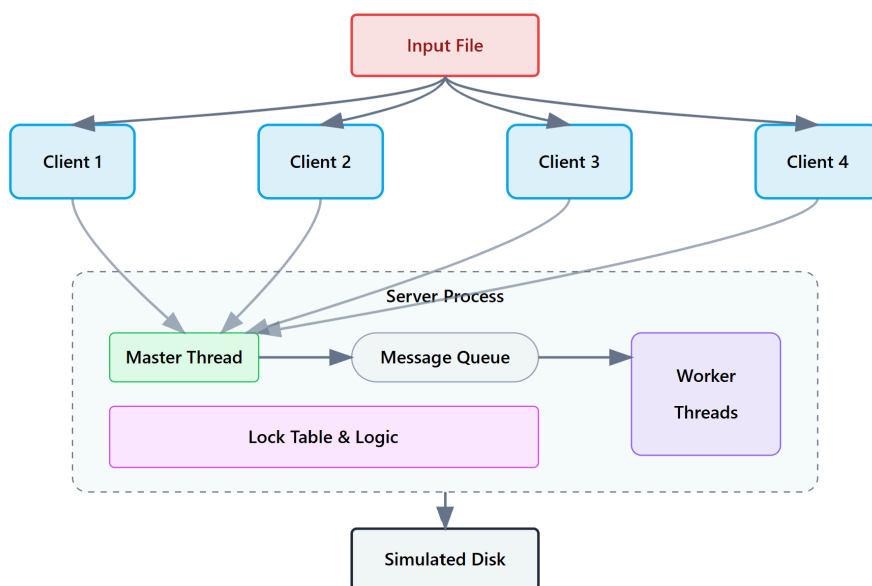
Assignment 6

Due 11:59 p.m., ~~April 3~~ Monday, April 6, 2026

IMPORTANT! This is an individual (or in pair) assignment. You may discuss broad issues of interpretation, understanding, and general approaches to a solution. However, the development of a specific solution or program code must be your own work. The assignment is expected to be entirely your own, designed and coded by you alone. If you need assistance, please consult your instructor or the TAs. Be sure to read the specific policies outlined in the [Academic Honesty in Computing](#) section.

Introduction

In this assignment, you will extend your implementation from Part 1 of Assignment 5 into the concurrent transaction processing system shown below. The system must support execution of multiple transactions concurrently while preserving correctness and consistency. All database operations must be coordinated through a central server which enforces concurrency control and manages disk access.



The implementation must use processes, sockets, threads, and synchronization primitives on a Unix/Linux system. Client processes must be created using `fork`, and all communication between clients and the server must be implemented using TCP sockets. The server must use a multithreaded design with a master thread and worker

threads. A shared message queue must be used within the server to coordinate request processing between threads. Synchronization among threads must be implemented using mutexes and condition variables to ensure safe access to shared data structures such as the lock table and request queue.

Transaction Properties

A *transaction* is a sequence of operations that must be executed as a single unit. In database systems, transactions are commonly described using the ACID (Atomicity, Consistency, Isolation, and Durability) properties:

- **Atomicity:** Every operation within a transaction is bound together as a single atomic unit; if any single step fails, the entire sequence is aborted and the database is restored to its pre-transaction state.
- **Consistency:** A transaction must transition the database from one valid state to another, strictly adhering to all structural rules, including data types, constraints, and relational triggers.
- **Isolation:** Even when multiple transactions execute concurrently, the system must ensure the final state is identical to the state that would have occurred if they had been run one at a time.
- **Durability:** Once a transaction is finalized (committed), its changes are recorded in non-volatile storage. The data must survive subsequent system crashes, power failures, or hardware reboots.

In this assignment, you are required to enforce Isolation through concurrency control. Concurrent transactions must not interfere with each other in a way that produces incorrect results. This must be achieved using *Strict Two-Phase Locking* (Strict 2PL). Atomicity and Durability are simplified. A transaction is considered complete when the commit operation is executed. You are not required to implement rollback, logging, or crash recovery. Consistency is ensured by correct execution of operations from Part 1 and proper enforcement of concurrency control.

System Architecture

The system consists of five processes: four client processes and one server process. Each client process reads transactions from a common input file and executes them sequentially. Transactions assigned to different clients may execute concurrently. A client process must not directly access the database or disk. All requests must be sent to the server. The server process performs scheduling, concurrency control, and all disk operations using the database system implemented in Part 1.

Client–Server Interface

Clients communicate with the server using the following operations:

```
/* Read the specified disk block into buffer for transaction tid.
   Returns 1 on success, 0 on failure. */
int Read(int tid, int blockno, char *buffer);

/* Write the contents of buffer to the specified disk block for transaction tid.
   Returns 1 on success, 0 on failure. */
int Write(int tid, int blockno, char *buffer);

/* Commit the transaction and release all locks held by tid. */
void Commit(int tid);
```

The Read operation requests a specified disk block to be read into the provided buffer. The Write operation requests the contents of the buffer to be written to the specified disk block. The Commit operation indicates the end of a transaction.

All communication between clients and the server must be implemented using TCP sockets. Clients must

establish a connection to the server and send requests corresponding to Read, Write, and Commit operations. Each request must be transmitted as a structured message containing the transaction identifier, operation type, and any associated data. The server must receive these requests, process them, and return an appropriate response to the requesting client. The response must indicate whether the operation was successful and, in the case of a read operation, must include the requested data.

You may use the following request and response formats:

```
#define READ_REQ    0
#define WRITE_REQ   1
#define COMMIT_REQ  2
#define SHUTDOWN    3
#define BLKSIZE     256      /* Size of a disk block in bytes */
#define DISKSIZE    256      /* Total number of disk blocks */

typedef struct {
    int type;          /* request type: READ_REQ, WRITE_REQ, etc. */
    int tid;           /* transaction identifier */
    int blockno;       /* disk block number */
    char buffer[BLKSIZE]; /* data buffer for read/write operations */
} request_t;

typedef struct {
    int status;        /* 1 = success, 0 = failure */
    char buffer[BLKSIZE]; /* returned data for read operations */
} response_t;
```

The database system implemented in Part 1 must be reused. The server is the only component that may invoke the following functions:

```
/* Read BLKSIZE bytes from disk block blocknum into buffer. */
void diskread(int blocknum, char *buffer);

/* Write BLKSIZE bytes from buffer to disk block blocknum. */
void diskwrite(int blocknum, char *buffer);
```

Clients must not directly call any database or disk functions. All high-level operations (such as insert, delete, update, select, and relational algebra operations) must be decomposed into sequences of block-level read and write operations. The server is responsible for translating these operations into calls to the underlying database system.

Concurrency Control

You must implement concurrency control using Strict 2PL. Locks are managed at the granularity of individual disk blocks. Two lock modes are supported:

```
#define READ_LOCK  0 /* shared lock: multiple transactions may hold */
#define WRITE_LOCK 1 /* exclusive lock: only one transaction may hold */
```

A transaction must acquire the appropriate lock before accessing any block. A write lock conflicts with both read and write operations from other transactions. A read lock only conflicts with write operations from other transactions. All locks (read and write) held by a transaction must be retained until the transaction officially commits. No locks may be released prematurely.

To prevent internal deadlocks (circular wait) between concurrent client threads, the system enforces a resource hierarchy:

- Transactions must always request locks in strictly ascending order of disk block numbers ($\text{Block}_1 < \text{Block}_2 < \text{Block}_3 \dots$).
- If a transaction currently holds a lock on a block (e.g., Block_{50}) and it requires a lower-numbered block (e.g., Block_2), it must release all currently held locks and restart its acquisition sequence beginning with the lowest block number.

The Lock Table. The server maintains a global lock table with one entry per disk block. Each entry manages the current state, the active holders, and a simple queue for waiting transactions to ensure fairness and prevent starvation.

```
typedef struct lock_request {
    int tid;                /* transaction requesting the lock */
    int mode;               /* READ_LOCK or WRITE_LOCK */
    struct lock_request *next; /* next request in waiting queue */
} lock_request_t;

typedef struct {
    int blockno;            /* disk block number */
    int lock_mode;          /* current lock mode (READ_LOCK, WRITE_LOCK, or FREE) */
    int holder_count;       /* number of transactions currently holding the lock */
    int holders[10];        /* array of transaction IDs holding the lock */
    lock_request_t *waiting; /* queue of waiting lock requests */
    pthread_cond_t cond;    /* condition variable to block/wake worker threads */
} lock_entry_t;

lock_entry_t lock_table[DISKSIZE]; /* one entry for every block in the simulated disk */
```

Lock Operations. The following functions must be implemented to manage thread synchronization:

```
/* Request a lock on block blockno for transaction tid in the given mode.
   Returns 1 if the lock is granted, otherwise blocks until available. */
int lock_acquire(int tid, int blockno, int mode);

/* Release the lock held by transaction tid on block blockno. */
void lock_release(int tid, int blockno);

/* Release all locks held by transaction tid at commit time. */
void release_all_locks(int tid);
```

Server Implementation

The server must be implemented as a multithreaded process using a thread pool. A fixed number of worker threads must be created at startup. Incoming requests from clients must be placed into a request queue implemented using a shared POSIX message queue and processed by worker threads. Each request is removed from the queue, scheduled according to the concurrency control mechanism, and executed by invoking the appropriate disk operations.

You may use the following message formats:

```
typedef struct {
    int type;                /* READ_REQ, WRITE_REQ, COMMIT_REQ, SHUTDOWN */
    int tid;                /* transaction identifier */
    int blockno;            /* disk block number */
    char buffer[BLKSIZE];   /* data buffer */
} mq_msg_t;
```

The server must ensure that requests are processed in a manner consistent with the concurrency control protocol and that all access to disk blocks is performed through the underlying database system.

Shared data structures, including the lock table and request queue, must be protected using mutexes. Condition variables must be used to coordinate access to shared resources. Threads must block when resources are unavailable and resume when they become available. Busy waiting is not permitted.

Transaction Execution

Each client reads transactions from the input file and executes them sequentially. A transaction consists of a sequence of operations followed by a commit command. Transactions assigned to different clients may execute concurrently.

Input Format. The input to the system consists of a sequence of transactions from multiple clients contained in a single file. Each transaction is identified by a transaction identifier of the form T_{ij} , where i denotes the client number and j denotes the transaction number for that client.

Each client process reads only those transactions whose identifiers begin with its client number. For example, client 1 processes transactions T11, T12, T13, and so on, while client 2 processes transactions T21, T22, etc. Each client maintains its own file pointer into the input file.

A transaction consists of a sequence of commands followed by a commit command. The format of commands within a transaction is the same as in Part 1, including DDL and DML operations such as CR, IN, RM, UP, PR, SL, PJ, UN, DF, and NJ. Each command appears on a new line, and any associated data appears on subsequent lines as defined in Part 1.

The end of a transaction is indicated by a line containing the transaction identifier followed by the command C. For example:

```
T11 CR flights 5 1
fid I
orig S
dest S
dist I
price I

T11 IN flights 2
4821 BOS MIA 1258 200
1934 BOS JFK 215 100

T11 C
```

Transactions from different clients may appear interleaved in the input file. Each client must ignore transactions that do not correspond to its identifier. Transactions assigned to a given client must be executed sequentially in the order they appear in the file. Transactions from different clients may execute concurrently.

High-level commands in a transaction must be decomposed into block-level read and write operations by the client and server. Concurrency control is applied at the level of disk blocks accessed during execution.

You may assume that the input is syntactically correct. Semantic checks required in Part 1 must still be enforced.

Commit Operation. When a transaction commits, all locks held by that transaction must be released:

```
void Commit(int tid) {
    release_all_locks(tid);
}
```

No logging or recovery mechanisms are required.

Termination. A client process terminates after completing all assigned transactions. Before termination, it must send a shutdown request to the server. The server terminates after all client processes have completed.

Dealing with Non-Existent Relations

To handle the race condition in which a client attempts to access a table before it has been initialized, the server must implement a retry protocol rather than immediately returning a “Relation Not Found” error. If a worker thread receives a request for a relation that is not currently present in the catalog, it should enter a synchronized wait state using a global condition variable.

Once any other worker thread successfully commits a CR operation, it must broadcast a signal to wake all waiting threads. By allowing a fixed number of retries before officially failing, the server ensures that the logical order of the input file is maintained even if the network packets from different client processes arrive out of sequence.

Programming Notes

- Write programs (`server.c`, `client.c`) to simulate a concurrent transaction processing system as described in the assignment.
- Compile the program with:

```
$ gcc -Wall -o server server.c -lpthread -lrt
$ gcc -Wall -o client client.c
```

- Run the program with:

```
$ ./server &
$ ./client 1 < data.in & // You may also exec() client processes from the server.
$ ./client 2 < data.in &
$ ./client 3 < data.in &
$ ./client 4 < data.in &
```

where `data.in` is the input file which contains the commands and data for the simulation. Use the format described in the assignment.

- Sample input: [data.in](#)

Handin

When completed, submit your C source, `server.c` and `client.c`, on the course website.

References

- *Concurrency Control and Recovery in Database Systems* by Philip Bernstein, Vassos Hadzilacos, and Nathan Goodman. Addison-Wesley, 1987.
- *Database System Concepts, Seventh Edition* by Avi Silberschatz, Henry F. Korth, S. Sudarshan. McGraw-Hill, 2019.
- *An Introduction to Database Systems, 8th Edition* by C. J. Date. Pearson/Addison-Wesley, 2004.

- **Welcome: Sean**

◦ [LogOut](#)

